OpenZeppelin | security

Era v.34 Upgrade Audit



October 29, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 6

Security Model and Trust Assumptions 8

Privileged Roles 8

Low Severity 10

L-01 Force-Failing a Transaction Earns the Operator More Than Executing It 10

L-02 Gateway Operator Can Force-Fail Relay Wrappers and Drop L1→L2 Transactions of Chains Settling on It 11

Notes & Additional Information 12

N-01 Effective L1→L2 Gas Limit Is Not Exposed 12

N-02 Events Do Not Include Airbender Commitments 13

N-03 getMemPointer Relies on Callers Having Validated the Offset 14

N-04 getL1ChainId does not validate the size of the returned data 14

Conclusion 16

Appendix 17

Issue Classification 17

Summary

Type	Layer 2 & Rollups
Timeline	From 2026-09-25 To 2026-09-30
Languages	Solidity

Total Issues	6 (4 resolved, 1 partially resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	2 (1 resolved)
Notes & Additional Information	4 (3 resolved, 1 partially resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin audited the changes that five pull requests to the matter-labs/era-contracts repository introduce. The pull requests form the v34-era protocol upgrade for EraVM chains. All five were merged into the `draft-v32-era-only` branch; the version was later renamed to v34-era, and the consolidated audit branch is `draft-v34-era-only`. Each pull request was reviewed as the diff between its base and head commits:

Pull request	Description	Base commit	Head commit
#2451	Require a Boojum and an Airbender proof for every Era batch	`1f6bcaf`	`905de5a`
#2450	Cap the gas of caller-supplied L1→L2 transactions at 15M	`3b1aa09`	`04487a4`
#2500	Allow the operator to force-fail L1→L2 transactions	`1f6bcaf`	`2b4652a`
#2505	Remove the Boojum PLONK verifier for Era chains	`bdc84f2`	`8b9cfad`
#2487	Validate memory offsets used to build zero-length pointers in the EVM emulator	`1f6bcaf`	`68f0054`

In scope were the following files:

#2451: 31 Solidity files (4 added, 27 modified)

```
l1-contracts/contracts/common/Config.sol
l1-contracts/contracts/common/L1ContractErrors.sol
l1-contracts/contracts/common/StateTransitionTypes.sol
l1-contracts/contracts/governance/IServerNotifier.sol
l1-contracts/contracts/governance/ServerNotifier.sol
l1-contracts/contracts/state-transition/ChainTypeManagerBase.sol
l1-contracts/contracts/state-transition/EraChainTypeManager.sol
l1-contracts/contracts/state-transition/IChainTypeManager.sol
l1-contracts/contracts/state-transition/ZKsyncOSChainTypeManager.sol
l1-contracts/contracts/state-transition/chain-deps/StoredBatchHashing.sol
l1-contracts/contracts/state-transition/chain-deps/ZKChainStorage.sol
l1-contracts/contracts/state-transition/chain-deps/facets/Admin.sol
l1-contracts/contracts/state-transition/chain-deps/facets/Committer.sol
l1-contracts/contracts/state-transition/chain-deps/facets/Executor.sol
l1-contracts/contracts/state-transition/chain-deps/facets/Getters.sol
```

```
l1-contracts/contracts/state-transition/chain-deps/facets/ZKChainBase.sol
l1-contracts/contracts/state-transition/chain-deps/gateway-ctm-deployer/
GatewayCTMDeployer.sol
l1-contracts/contracts/state-transition/chain-deps/gateway-ctm-deployer/
GatewayCTMDeployerCTMBase.sol
l1-contracts/contracts/state-transition/chain-deps/gateway-ctm-deployer/
GatewayCTMDeployerVerifiers.sol
l1-contracts/contracts/state-transition/chain-interfaces/IAdmin.sol
l1-contracts/contracts/state-transition/chain-interfaces/ICommitter.sol
l1-contracts/contracts/state-transition/chain-interfaces/IEraDualVerifier.sol
l1-contracts/contracts/state-transition/chain-interfaces/IEraMultiProofVerifier.sol
(added)
l1-contracts/contracts/state-transition/chain-interfaces/IExecutor.sol
l1-contracts/contracts/state-transition/chain-interfaces/IGetters.sol
l1-contracts/contracts/state-transition/libraries/BatchDecoder.sol
l1-contracts/contracts/state-transition/verifiers/EraDualVerifier.sol
l1-contracts/contracts/state-transition/verifiers/EraMultiProofTestnetVerifier.sol
(added)
l1-contracts/contracts/state-transition/verifiers/EraMultiProofVerifier.sol (added)
l1-contracts/contracts/state-transition/verifiers/EraTestnetVerifier.sol
l1-contracts/contracts/upgrades/EraSettlementLayerV33Upgrade.sol (added)
```

#2450: 3 Solidity files (modified)

```
l1-contracts/contracts/common/Config.sol
l1-contracts/contracts/state-transition/chain-deps/facets/Mailbox.sol
l1-contracts/contracts/state-transition/chain-interfaces/IGetters.sol
```

#2500: no production Solidity files; the change is in the Yul bootloader

```
system-contracts/bootloader/bootloader.yul
```

#2505: 4 Solidity files (modified)

```
l1-contracts/contracts/common/Config.sol
l1-contracts/contracts/common/StateTransitionTypes.sol
l1-contracts/contracts/state-transition/chain-deps/gateway-ctm-deployer/
GatewayCTMDeployerVerifiers.sol
l1-contracts/contracts/state-transition/verifiers/EraMultiProofVerifier.sol
```

#2487: no production Solidity files; the change is in the Yul EVM emulator

```
system-contracts/contracts/EvmEmulator.yul
system-contracts/evm-emulator/EvmEmulatorFunctions.template.yul
system-contracts/evm-emulator/EvmEmulatorLoop.template.yul
```

System Overview

The v34-era upgrade hardens the security of EraVM chains by settling every batch behind two independent proof systems, and adds safeguards to keep chains live while the new proof system is rolled out.

Multi-proof verification (#2451, #2505). Every Era batch now carries two commitments: the existing Boojum commitment and a new Airbender commitment. The two share the pass-through data (state root and enumeration index), the metadata hash, the system logs, the state-diff hash and the blob commitments. They differ only in the auxiliary output: Airbender hashes the bootloader heap with Blake2s and sets the events-queue hash to zero. `StoredBatchInfo` gains an `airbenderCommitment` field, which the Committer derives at commit time from the new `CommitBatchInfo.airbenderBootloaderHeapHash`.

When a batch is proven, the Executor builds one public input per proof system, and each chains on that system's own commitments. The last batch before the upgrade has no Airbender commitment, so its Boojum commitment is used as the Airbender predecessor; new chains carry a genesis Airbender commitment in `ChainCreationParams`. The new `EraMultiProofVerifier` accepts a fixed-length envelope, `[4, boojum(24 words), airbender(44 words)]`, and requires a valid proof from every proof system it has a verifier for, unless the chain has disabled that system. #2505 removes the Boojum PLONK route and `EraDualVerifier`, so the multi-proof verifier wraps the FFLONK verifier directly.

Chain admins can disable one proof system, but never both, with `setProofSystemStatus`, to keep a chain live through a prover incident. The setting is read when a batch is proven, so it can be changed while committed batches are waiting. The commit, prove and execute encoding version moves to 5 (6 for ZKsync OS commits), and the `EraSettlementLayerV33Upgrade` contract requires every committed batch to be executed before the upgrade.

Gas cap for caller-supplied L1→L2 transactions (#2450). Priority transactions must eventually be included and can't be split across batches, so the work each one can impose has to be bounded at admission. On EraVM chains, the body gas that a caller can request for an L1→L2 transaction is now capped at 15M, or at the chain's `priorityTxMaxGasLimit` if that is lower. Protocol-authored transactions, i.e. service transactions and Gateway relay wrappers, keep the chain limit, and ZKsync OS chains are exempt. The 15M cap appears reasonable given current usage of L1→L2 transactions. However, the audit cannot guarantee that it is

sufficient for every possible transaction, and any operation that needs more body gas can no longer be submitted, or forced, through L1.

Force-failing L1→L2 transactions (#2500). The operator can mark a priority transaction as failed before any of its code runs, by setting a new bit in the transaction metadata the bootloader reads. A force-failed transaction takes the same path as one that reverted on its first instruction: the deposited value is refunded minus fees, a failure status is logged, and the priority queue moves past it. The bootloader also emits a marker log so that a forced failure can be told apart from a natural one. The bit is only accepted on priority transactions, and only when the batch settles on L1.

This changes the censorship-resistance guarantees of EraVM chains. Priority transactions are consumed from the queue strictly in order, and the operator cannot skip one. Before this upgrade that made censorship all-or-nothing: to avoid executing a single L1→L2 transaction, the operator had to stop consuming the queue at that transaction, which blocked every deposit and every forced transaction queued behind it. Denying one user meant denying all of them, visibly, for as long as the censorship lasted — a cost the operator could not isolate to its target. The force-fail bit removes that constraint. A force-failed transaction counts as processed, so the queue advances past it and the transactions behind it are unaffected. The operator can now deny one chosen transaction, including a forced withdrawal, while the chain continues to operate normally and every other user sees nothing wrong. No L1 contract reads the marker log, and none limits how often the bit is used, so nothing on L1 bounds the practice or makes it costly; detection depends on off-chain monitoring. There is also no automatic remedy on these chains: Priority Mode, which lets anyone settle batches once a transaction has gone unprocessed, is restricted to ZKsync OS and has never been available on EraVM. Deposits made through the bridges can still be reclaimed with claimFailedDeposit, but funds already on L2 can no longer be withdrawn by force through L1.

EVM emulator fix (#2487). For memory accesses with size zero, the EVM emulator accepted any offset, as the EVM does, but still used that offset to build EraVM heap pointers. A pointer could exceed 32 bits or wrap around into the emulator's own memory. The fix ignores the offset whenever the size is zero, and adds a width check when call parameters are packed.

Security Model and Trust Assumptions

During the audit, the following assumptions were made:

- **Off-chain components.** The server, the Boojum prover and the Airbender guest program are compatible with the new encoding versions and the multi-proof envelope. The two EraVM implementations produce identical shared outputs for every valid batch; any disagreement stops proving until one system is disabled.
- **Airbender predecessor binding.** The Airbender guest takes the predecessor's metadata and auxiliary output hashes as free witnesses and binds only its state root. This is what makes it safe to use the Boojum commitment as the predecessor at the upgrade boundary, and to resume after a period in which Airbender was disabled.
- **Governance-supplied genesis.** The `genesisAirbenderBatchCommitment` in the chain creation parameters is correct. The contracts check only that it is non-zero.
- **Gateway.** The Matter Labs team stated that they do not plan to run a Gateway or migrate EraVM chains onto one, and that the Gateway-related checks exist for consistency. Issues that require a running Gateway are therefore reported as latent.
- **ZKsync OS chains** are not upgraded as part of this version.
- **Operator censorship of L1→L2 transactions.** The force-fail mechanism of #2500 gives the operator the power to fail any L1→L2 priority transaction. No on-chain check limits it, and the forced failure is recorded only in a log that L1 does not read. Users' ability to get L1→L2 transactions executed, and to exit through L1, therefore relies on the operator not abusing this power.

Privileged Roles

- **Chain admin:** can disable or re-enable either proof system with `setProofSystemStatus`, including while priority mode is active. Batches proven while a system is disabled are never verified by it afterwards, so the guarantee that two independent systems verify every batch holds only for batches proven while both were enabled.
- **Operator / validator (batch producer):** decides which priority transactions to force-fail, and how much overhead to charge each transaction. The force-fail power is new in this

upgrade. It lets validators censor any L1→L2 transaction, including forced withdrawals, and they are paid the transaction's full gas limit when they use it.

- **Governance (CTM and Bridgehub owners):** registers the verifier for each protocol version, sets the chain creation parameters (including the genesis Airbender commitment), sets each chain's `priorityTxMaxGasLimit`, and decides whether any Gateway is whitelisted as a settlement layer.

Low Severity

L-01 Force-Failing a Transaction Earns the Operator More Than Executing It

When the `forceFail` bit is set, `ZKSYNC_NEAR_CALL_executeL1Tx` calls `nearCallPanic()` before the transaction runs. Unlike a normal return or revert, a near-call panic burns all the gas passed to the frame. In `processL1Tx`, `gasSpentOnExecution` is therefore the full `gasForExecution`.

The refund is calculated as `(reservedGas + gasForExecution) - (gasSpentOnExecution + ergsSpentOnPubdata)`, floored at zero. With `gasSpentOnExecution` equal to `gasForExecution`, the two execution terms cancel, and the refund reduces to `reservedGas - ergsSpentOnPubdata`. `reservedGas` is the part of the gas limit above the operator's trusted limit, which is at least `MAX_GAS_PER_TRANSACTION`. It is normally zero for L1→L2 transactions, so the calculated refund, `potentialRefund`, is zero and none of the execution budget is returned. The final refund is `refundGas = max(operatorRefund, potentialRefund)`. Here `operatorRefund` is a value the operator supplies through the `askOperatorForRefund` hook, and the bootloader only lets it raise the refund, never lower it. With `potentialRefund` at zero, the final refund is whatever the operator chooses, and nothing requires it to be more than zero. The operator is then paid `gasPrice × (gasLimit - refundGas)`, which is close to the transaction's entire `gasLimit`, and the user's refund recipient receives the deposit minus that fee.

A transaction that executed normally would have its unused execution gas refunded and pay only for what it used. The operator therefore earns more by force-failing a transaction than by executing it, even though a force-failed transaction does no work. This gives the party that decides whether to force-fail a financial incentive to do so, and the user pays in full for a transaction that never ran.

In addition, the marker log that `processL1Tx` emits for each force-failed transaction is not charged to the transaction. The status log is covered by the fixed intrinsic pubdata cost, `L1_TX_INTRINSIC_PUBDATA`, which is 88 bytes (one L2→L1 log) and is deducted up front. The marker, however, is emitted after the fee is settled and is covered by neither the intrinsic cost nor the measured pubdata. Each force-failed transaction therefore emits 88 bytes of

pubdata that it does not pay for. This is negligible compared to the fee the operator collects for such a transaction.

Consider refunding the execution gas (`gasForExecution`) of force-failed transactions, so that the operator is paid only for preparation and pubdata and gains nothing by force-failing a transaction. Additionally, consider charging the marker log's pubdata to the force-failed transaction.

Update: *Acknowledged, not resolved. The team stated:*

> *Acknowledged, we will not provide a fix. Force-failing is meant to be priced exactly like an L1→L2 transaction that reverts on its first instruction. Such a revert also ends in `nearCallPanic` and burns the transaction's whole execution budget, so a force-failed transaction pays what a reverted one would. Force-fail is an operator escape hatch for priority operations that can't be processed. Every use is visible on L1 through the marker log, and bridged funds can still be recovered with `claimFailedDeposit`.*
>
> *We do not consider a meaningful risk that the operator can get additional profits with force-failing the transactions. The operator is already trusted with liveness and transaction ordering, so the extra fee it could collect this way doesn't add meaningful risk.*
>
> *The marker log's uncharged pubdata (88 bytes) is negligible next to that fee.*

L-02 Gateway Operator Can Force-Fail Relay Wrappers and Drop L1→L2 Transactions of Chains Settling on It

The `processTx` function of the bootloader accepts the new `forceFail` bit for any priority transaction, provided the batch settles on L1. It does not check who sent the transaction. Gateway settles on L1, so its operator can force-fail any priority transaction in a Gateway batch. This includes the relay wrappers, sent by `SETTLEMENT_LAYER_RELAY_SENDER`, that deliver L1→L2 transactions to chains settling on Gateway (`requestL2TransactionToGatewayMailboxWithBalanceChange`).

A force-failed wrapper never executes `InteropCenter.forwardTransactionOnGatewayWithBalanceChange`. The transaction hash is therefore never added to the destination chain's priority tree on Gateway, and the chain's balance is never credited, so the chain can never execute the transaction. Since no status log for it is ever emitted, the user cannot reclaim the deposit through

`L1Nullifier`, and the funds remain locked. Once a later wrapper succeeds, the chain's L1 and Gateway priority trees diverge, and `Migrator.forwardedBridgeMint` reverts with `NotHistoricalRoot` on any attempt to migrate the chain back to L1. No mechanism exists to re-send the wrapper.

The Matter Labs team has stated that they do not plan to run a Gateway. The issue is therefore latent: it requires governance to whitelist a Gateway settlement layer (`L1Bridgehub.setSettlementLayerStatus`), but it would have high impact if a Gateway were enabled.

Consider rejecting the `forceFail` bit in `processTx` when the priority transaction's sender (`getFrom`) is `SETTLEMENT_LAYER_RELAY_SENDER` or `SERVICE_TRANSACTION_SENDER`, so that protocol-authored transactions can never be force-failed. Alternatively, consider documenting that this restriction must be in place before any Gateway is whitelisted as a settlement layer.

Update: *Resolved in pull request #2560. The team stated:*

> *Added comments about this behavior. Note: operator can also force-fail general service transactions, e.g.* `allowEvmEmulation`*. We don't consider it an important issue.*

Notes & Additional Information

N-01 Effective L1→L2 Gas Limit Is Not Exposed

The `getPriorityTxMaxGasLimit` function in `Getters` returns the per-chain `priorityTxMaxGasLimit` value, which is seeded at `72,000,000`. Caller-supplied L1→L2 transactions on EraVM chains are validated against a lower limit than that: `MailboxFacet._requestL2Transaction` passes the lower of the chain value and `USER_PRIORITY_TX_MAX_GAS_LIMIT`, which is `15,000,000`.

The getter therefore reports a limit that caller-supplied transactions cannot reach. On a chain left at the default it returns `72,000,000`, while the chain accepts at most `15,000,000` of transaction body gas from a caller. Integrators that size a deposit from either value build transactions that revert with `TooMuchGas`. The interface notes the distinction in a comment, but no function returns the effective value.

Consider adding an external getter that returns the effective caller-supplied limit, so that integrators can read the value their transactions are validated against.

Update: *Resolved in pull request #2559. The team stated:*

> *A new `getUserPriorityTxMaxGasLimit()` has been added, returning the caller-supplied limit L1→L2 transactions are actually checked against.*

N-02 Events Do Not Include Airbender Commitments

The upgrade adds an Airbender commitment to the data that is hashed into stored batch hashes, but two events used to rebuild that data off-chain do not emit it:

- **`BlockCommit`**. `StoredBatchInfo` now includes an `airbenderCommitment` field, which is part of the stored batch hash. However, when an Era batch is committed, the `BlockCommit` event emits only the batch number, batch hash and Boojum `commitment` (`ICommitter.sol`). Off-chain tools that rebuild `StoredBatchInfo` from events, such as indexers, therefore cannot obtain `airbenderCommitment` from them. Without it, the rebuilt struct fails `_checkBatchHashMismatch` when used as the predecessor of the next commit or proof. The value can only be recomputed by decoding the commit calldata.
- **`NewChainCreationParams`**. The `_processValidatedChainCreationParams` function builds the genesis batch with `airbenderCommitment` set to the new `genesisAirbenderBatchCommitment` parameter, and stores its hash as `storedBatchZero`, which every new chain installs as `storedBatchHashes[0]`. For Era chains this value must be non-zero (`EraChainTypeManager.sol`). However, the `NewChainCreationParams` event emits `genesisBatchCommitment` but not `genesisAirbenderBatchCommitment`, even though `newChainCreationParamsBlock` records the event's block for off-chain use. A tool that rebuilds batch zero from the event cannot recover the field. Defaulting it to zero produces a different hash, so the first commit reverts with `BatchHashMismatch`. The value can only be obtained from the `setChainCreationParams` calldata or the genesis configuration.

Consider emitting `airbenderCommitment` in `BlockCommit`, or in a separate event, and adding `genesisAirbenderBatchCommitment` to `NewChainCreationParams`, so that the stored batch data can be rebuilt from logs alone. Since `BlockCommit` already has three indexed parameters, the new field would have to be non-indexed.

Update: *Partially Resolved in pull request #2559. The team stated:*

> *Added `BatchAirbenderCommitment(batchNumber, airbenderCommitment)` event per Era commit instead of extending `BlockCommit`, since changing its signature would move `topic0` for indexers. The genesis half (a `NewChainCreationParams` companion event) is acknowledged, not fixed: emitting it pushed `EraChainTypeManager` 4 bytes over EIP-170, and the value is available from the `setChainCreationParams` calldata and the genesis config.*

N-03 `getMemPointer` Relies on Callers Having Validated the Offset

For a non-zero size, the `getMemPointer` function of the EVM emulator returns `add(MEM_OFFSET(), rawOffset)` without bounding `rawOffset`. It relies on the caller having already called `expandMemory` or `expandMemory2` on the same `(offset, size)` pair, which panics above `MAX_POSSIBLE_MEM_LEN`. All current call sites do this. However, a future call site that doesn't would build an unbounded heap pointer again, which is the bug this PR fixes.

Consider bounding the offset inside `getMemPointer`, for example by panicking when `rawOffset` exceeds `MAX_POSSIBLE_MEM_LEN` in the non-zero-size branch. Alternatively, consider documenting in the function's NatSpec that callers must call `expandMemory` or `expandMemory2` with the same `(offset, size)` beforehand.

Update: *Resolved in pull request #2559. The team stated:*

> *`getMemPointer` now panics if the offset is above `MAX_POSSIBLE_MEM_LEN`.*

N-04 `getL1ChainId` does not validate the size of the returned data

The `getL1ChainId` function of the bootloader writes a selector to scratch memory, calls the `L2AssetTracker` contract with `staticcall`, and returns the first memory word. It checks that the call succeeded but not that 32 bytes were returned. A call that succeeds with less return data, such as a call to an address with no deployed code, leaves the output area untouched, so the function returns the selector it wrote rather than a chain ID.

The value is used by the force-fail guard in `processTx`, which compares it against the settlement layer chain ID. That chain ID is read from bootloader memory supplied by the

operator, so the comparison can be made to pass. The batch still cannot be committed: the same value is sent to L1 as a system log, and the CommitterFacet contract requires it to equal block.chainid. The guard is therefore sound, but only because of a check made elsewhere.

The bootloader validates `returndatasize` where it calls untrusted code, for example in `ensureCorrectAccountMagic`, and omits the check where it calls system contracts, as in `getRawCodeHash`. The `getL1ChainId` function follows the latter pattern while gating a security-relevant capability.

Consider checking that `returndatasize()` equals 32 after the call and raising an assertion error otherwise, so that the returned value is rejected unless the call produced a full word.

Update: *Resolved in pull request #2559.*

Conclusion

The audited pull requests (#2451, #2450, #2500, #2505 and #2487) make up the v34-era upgrade. It requires every EraVM batch to be proven by both Boojum and Airbender, and adds liveness safeguards for the rollout of Airbender. It also removes the Boojum PLONK verifier and fixes an EVM-equivalence bug in the EVM emulator. The core multi-proof design was found to be sound.

The most significant concerns are related to the new trust model coming with the force-fail mechanism. It allows the operator to block any L1→L2 transaction including forced withdrawals. It also pays the operator more for failing a transaction than for executing it. Other notable findings include latent Gateway issues, which could lock deposits of chains settling on a Gateway.

Each pull request came with an extensive description of its motivation and design, and the Matter Labs team answered our questions promptly and in detail.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.